Projet 2 — Prétraitement de CIC-DDoS2019

Des fichiers CSV bruts à des jeux train/test prêts pour l'apprentissage, sans fuite de données

Tjeck Tjeck Thomas Noel — Maîtrise en informatique, Université du Québec en Outaouais (UQO) — CYB6063 — 19 octobre 2026

Élément	Choix retenu
Source des données	Version Kaggle « CIC-DDoS2019 30GB Full Dataset CSV Files » (rodrigorosasilva), copie des 18 CSV officiels du Canadian Institute for Cybersecurity (UNB). Téléchargée avec `kagglehub` dans Google Colab.
Échantillonnage	Lecture par morceaux de 500 000 lignes ; taux de tirage par classe = min(1 ; 50 000 / effectif réel). BENIGN, UDPLag et WebDDoS conservés en entier. Résultat : 916 769 flux sur 70 427 637 (1,3 %).
Reproductibilité	random_state = 42 et np.random.seed(42) partout. Python 3.13.16, pandas 2.2.3, NumPy 2.1.3, scikit-learn 1.6.1, imbalanced-learn 0.14.2, matplotlib 3.10.0, seaborn 0.13.2, joblib 1.6.0, pyarrow 23.0.1.
Livrables	Notebook Projet2_Pretraitement_CICDDoS2019_TjeckTjeckThomasNoel.ipynb (parties A à F), pipeline joblib, train.parquet et test.parquet. Le code est résumé en annexe.

Partie A — Compréhension du jeu de données (6 pts)

Question 1 — Présentation de CIC-DDoS2019 (2 pts)

Énoncé : origine, objectif, CICFlowMeter, jours d'enregistrement, fichiers CSV et tailles ; comparaison avec NSL-KDD.

CIC-DDoS2019 a été publié en 2019 par le Canadian Institute for Cybersecurity de l'Université du Nouveau-Brunswick (Sharafaldin et al., 2019). Son objectif est de fournir un jeu réaliste et récent pour entraîner et évaluer des détecteurs d'attaques DDoS, en particulier les attaques modernes par **réflexion** et par **exploitation**, absentes des anciens jeux. Le trafic du banc d'essai (PCAP) a été converti en flux bidirectionnels par **CICFlowMeter-V3**, qui calcule plus de 80 caractéristiques par flux (durée, nombre et tailles de paquets, débits, drapeaux TCP, temps inter-arrivées...) et ajoute un label. Deux journées ont été enregistrées : le **12 janvier 2019** (jour d'entraînement, 12 attaques) et le **11 mars 2019** (jour de test, 7 attaques). Les 18 fichiers CSV pèsent environ **31 Go** : 11 fichiers pour le 12 janvier (≈ 22 Go, dont TFTP.csv à lui seul 9,3 Go) et 7 pour le 11 mars (≈ 9 Go).

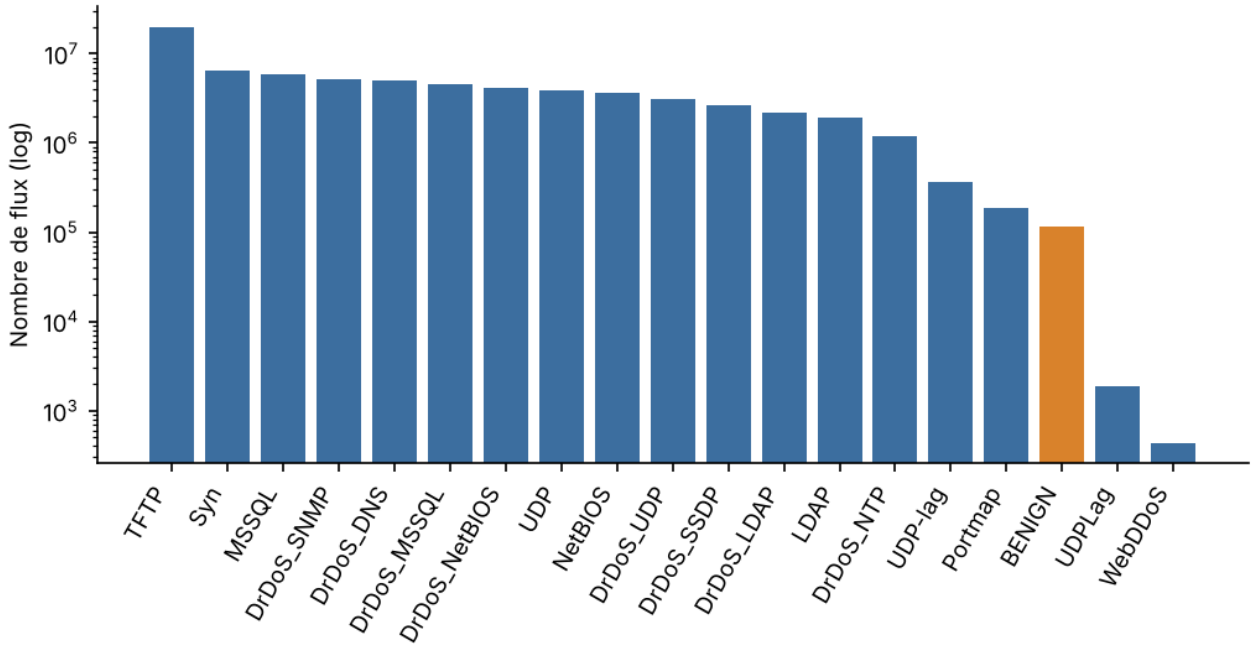
	CIC-DDoS2019	NSL-KDD
Nature des données	Flux réseau extraits de trafic capturé en 2019 (banc d'essai réaliste, profils B-Profile)	Connexions issues de la simulation DARPA 1998 (version nettoyée de KDD'99)
Nombre de features	Plus de 80 (CICFlowMeter), 88 colonnes dans les CSV	41
Types d'attaques	DDoS modernes : réflexion (DNS, LDAP, NTP, SSDP…) et exploitation (SYN, UDP, UDP-Lag)	4 catégories génériques : DoS, Probe, R2L, U2R

Question 2 — Taxonomie des attaques (4 pts)

Énoncé : lister les valeurs de Label, distinguer réflexion et exploitation, diagramme en barres en échelle log, justifier l'échelle log.

Les effectifs ont été calculés sur les **fichiers complets** en ne lisant que la colonne Label (70 427 637 flux, 19 valeurs). Le préfixe *DrDoS* signifie « Distributed Reflection DoS ». Une même attaque porte parfois deux noms selon le jour (DrDoS_LDAP / LDAP, UDP-lag / UDPLag…), ce qui est traité en Q11.

Figure 1 – Distribution des 19 labels du jeu complet (70,4 M flux, échelle log)



Catégorie	Principe	Labels
Réflexion	L'attaquant usurpe l'IP de la victime et interroge des serveurs tiers légitimes, qui renvoient des réponses souvent amplifiées vers la victime ; l'attaquant reste caché.	DrDoS_DNS, DrDoS_LDAP / LDAP, DrDoS_MSSQL / MSSQL, DrDoS_NTP, DrDoS_NetBIOS / NetBIOS, DrDoS_SNMP, DrDoS_SSDP, TFTP, Portmap
Exploitation	L'attaque abuse d'une faiblesse d'un protocole pour épuiser directement les ressources de la victime.	Syn (poignée de main TCP), DrDoS_UDP / UDP (UDP flood), UDP-lag / UDPLag, WebDDoS
Trafic normal	—	BENIGN

Pourquoi l'échelle logarithmique ? Les classes sont extrêmement déséquilibrées : TFTP représente 28,5 % des flux (20,1 millions), alors que WebDDoS n'en compte que 439, soit environ 45 700 fois moins (plus de 4 ordres de grandeur). BENIGN ne représente que 0,16 % des flux. En échelle linéaire, BENIGN, Portmap, UDPLag et WebDDoS seraient invisibles ; l'échelle log donne la même hauteur à chaque puissance de 10. Conséquence : l'accuracy seule serait trompeuse (prédire toujours « attaque » donnerait 99,8 %). Il faudra suivre le F1 et le rappel par classe, et rééquilibrer.

Partie B — Chargement et exploration (5 pts)

Question 3 — Chargement efficace (2 pts)

Énoncé : lecture par morceaux ou échantillonnage, conversion float64 → float32 et int64 → int32, mémoire avant/après avec memory_usage(deep=True).

Le jeu complet (31 Go) dépasse la RAM de Colab. Chaque CSV est lu par morceaux de 500 000 lignes, et chaque ligne est gardée avec une probabilité **min(1 ; 50 000 / effectif réel de sa classe)**, calculée à partir des effectifs de la Q2. On obtient environ 50 000 flux par attaque, en gardant BENIGN (113 828), UDPLag (1 873) et WebDDoS (439) en entier. Un tirage uniforme de 10 % aurait conservé le déséquilibre (≈ 2 M de flux TFTP contre 44 WebDDoS).

```
taux = {lab: min(1.0, 50_000 / n) for lab, n in labels.items()}; taux["BENIGN"] = 1.0
for chunk in pd.read_csv(f, chunksize=500_000, low_memory=False):
    p = chunk["Label"].str.strip().map(taux).to_numpy()
    morceaux.append(chunk[rng.random(len(chunk)) < p])      # tirage stratifié, graine 42
for col in df.select_dtypes("float64").columns: df[col] = df[col].astype("float32")
for col in df.select_dtypes("int64").columns:   df[col] = df[col].astype("int32")
```

Résultat : 916 769 flux × 88 colonnes. La mémoire passe de **955,4 Mo à 654,7 Mo (−31,5 %)**. Le gain reste sous 50 % car seules les 82 colonnes numériques (45 float64, 37 int64) sont réduites de moitié : les 6 colonnes texte (Flow ID, IP, Timestamp, SimillarHTTP, Label) pèsent à elles seules environ 350 Mo. La perte de précision du float32 est négligeable pour des débits ou des compteurs. L'échantillon est ensuite mis en cache au format Parquet dans Google Drive, pour éviter de relire 31 Go (environ 25 min) à chaque session.

Question 4 — Nettoyage des noms de colonnes (1 pt)

Énoncé : supprimer les espaces initiaux, snake_case, traiter les colonnes en double (ex. Fwd Header Length.1).

Les espaces initiaux (« Label ») ont été retirés dès le chargement par `str.strip()`. Les noms sont convertis en snake_case (`Flow Bytes/s` → `flow_bytes_s`), ce qui les rend homogènes et compatibles avec Parquet. Les doublons sont détectés par le nom (suffixe `.1`) et par le contenu (hachage de chaque colonne, puis `equals`). Les colonnes constantes, « égales » entre elles car toutes nulles, sont volontairement exclues de cette recherche : elles relèvent de la Q8. **6 doublons supprimés, 88 → 82 colonnes :**

Copie supprimée	Original conservé	Explication
Fwd Header Length.1	Fwd Header Length	Colonne écrite deux fois par CICFlowMeter
Avg Fwd / Bwd Segment Size	Fwd / Bwd Packet Length Mean	Même calcul (taille moyenne des paquets)
Subflow Fwd / Bwd Packets	Total Fwd / Backward Packets	Un seul sous-flux par flux
RST Flag Count	Fwd PSH Flags	Valeurs strictement identiques (artefact de l'outil)

Question 5 — Colonnes identifiantes (2 pts)

Énoncé : identifier les colonnes qui ne décrivent pas le comportement du flux, expliquer la fuite de données, les supprimer.

Huit colonnes décrivent la **construction du banc d'essai** et non le comportement du flux : `unnamed_0` (index), `flow_id`, `source_ip`, `destination_ip`, `source_port`, `destination_port`, `timestamp` et `simillar_http`. Les données confirment la fuite : **99,8 %** des 1 577 IP sources et **100 %** des 912 204 horodatages n'apparaissent qu'avec un seul label (chaque attaque a été jouée dans sa plage horaire, depuis des machines fixes). Le port source dominant identifie aussi la famille d'attaque (900 → LDAP, 634 → NTP, 564 → DNS, 80 → WebDDoS, 443 → BENIGN). Un modèle s'en servirait comme raccourci et obtiendrait un score presque parfait en test, mais il échouerait sur un autre réseau, avec d'autres IP et d'autres horaires. Un IDS doit apprendre des statistiques de flux généralisables. On garde `protocol` (vraie propriété du flux) et `inbound`. **Résultat : 82 → 74 colonnes, 633,7 → 319,4 Mo (−49,6 %)**. Seule la cible reste au format texte.

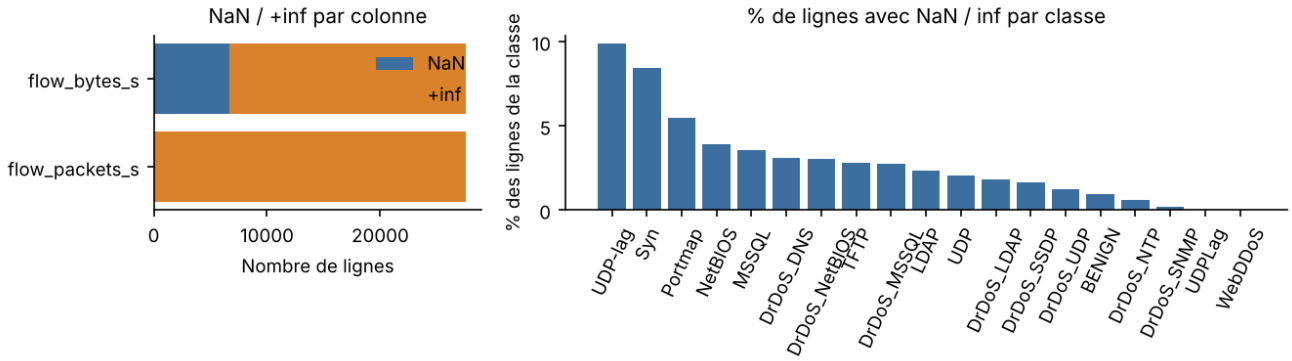
Partie C — Qualité des données (9 pts)

Question 6 — Valeurs manquantes et infinies (3 pts)

Énoncé : compter NaN et ±inf par colonne, justifier par un graphique, choisir et justifier une stratégie.

Seules **2 colonnes sur 73** sont touchées : `flow_bytes_s` (6 653 NaN + 20 928 +inf) et `flow_packets_s` (27 581 +inf). Il n'y a aucun −inf. Au total, **27 581 lignes (3,01 %)** sont concernées, et **100 %** d'entre elles ont `flow_duration = 0`. CICFlowMeter divise par une durée nulle : 0/0 donne NaN et x/0 donne +inf.

Figure 2 – Valeurs manquantes et infinies (Q6)



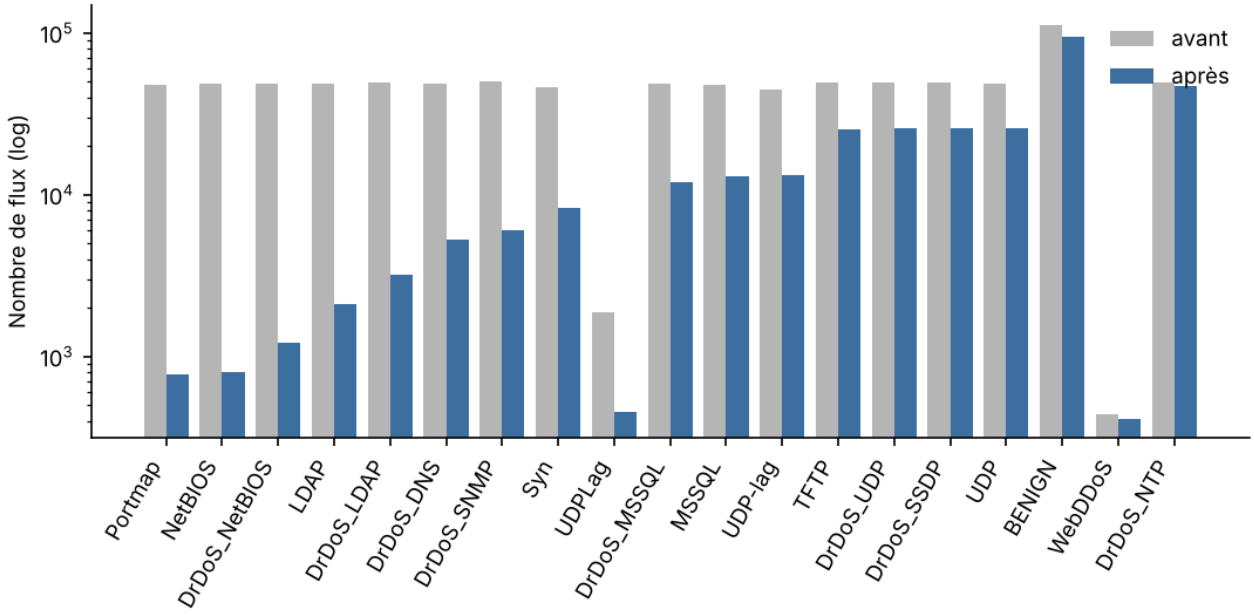
Stratégie : suppression des lignes (916 769 → 889 188, plus aucun NaN ni inf). (1) Le volume est faible : 3 % au total, au plus 9,9 % pour UDP-lag et 8,4 % pour Syn, qui gardent toutes plus de 45 000 exemples. UDPLag et WebDDoS ne perdent rien. (2) Un débit infini n'est pas une mesure ; une valeur inventée ajouterait du bruit. (3) Une imputation par la médiane devrait être ajustée sur le train seul (sinon fuite) et remplacerait des débits extrêmes, typiques des attaques courtes, par une valeur « normale ». L'alternative « +inf → maximum fini » reste défendable, mais la suppression est plus simple et sans risque de fuite.

Question 7 — Doublons (2 pts)

Énoncé : compter, supprimer, vérifier la nouvelle taille, tableau avant/après par classe, commenter.

578 126 lignes dupliquées (65 %) : le jeu passe de **889 188 à 311 062 lignes**, et il ne reste aucun doublon.

Figure 3 – Flux par classe avant / après suppression des doublons (Q7)



Les classes qui perdent le plus sont les réflexions à paquets fixes : Portmap (−98,4 %), NetBIOS (−98,3 %), DrDoS_NetBIOS (−97,5 %), LDAP (−95,7 %), DNS (−89,2 %), SNMP (−87,9 %). L'outil d'attaque envoie des millions de requêtes identiques (même taille, un paquet par flux, durée nulle). Une fois les IP, ports et horodatages retirés (Q5), ces flux deviennent des copies exactes. Syn, MSSQL et UDP-lag perdent 70 à 82 %, et TFTP, SSDP et UDP environ 48 %. À l'inverse, BENIGN (−15 %), WebDDoS (−6 %) et NTP (−5 %), dont les tailles et durées sont très variables, perdent peu. On trouve aussi **11 750 groupes de flux identiques portant des labels différents** (ex. NetBIOS / DrDoS_NetBIOS / Portmap), impossibles à séparer en multiclasse ; d'où l'harmonisation (Q11) et les familles (Q15). La suppression rend le jeu plus petit mais honnête : sans elle, une même ligne pourrait se trouver à la fois dans le train et dans le test.

Question 8 — Colonnes constantes ou quasi constantes (2 pts)

Énoncé : détection par nunique() ou VarianceThreshold, liste des colonnes supprimées, justification.

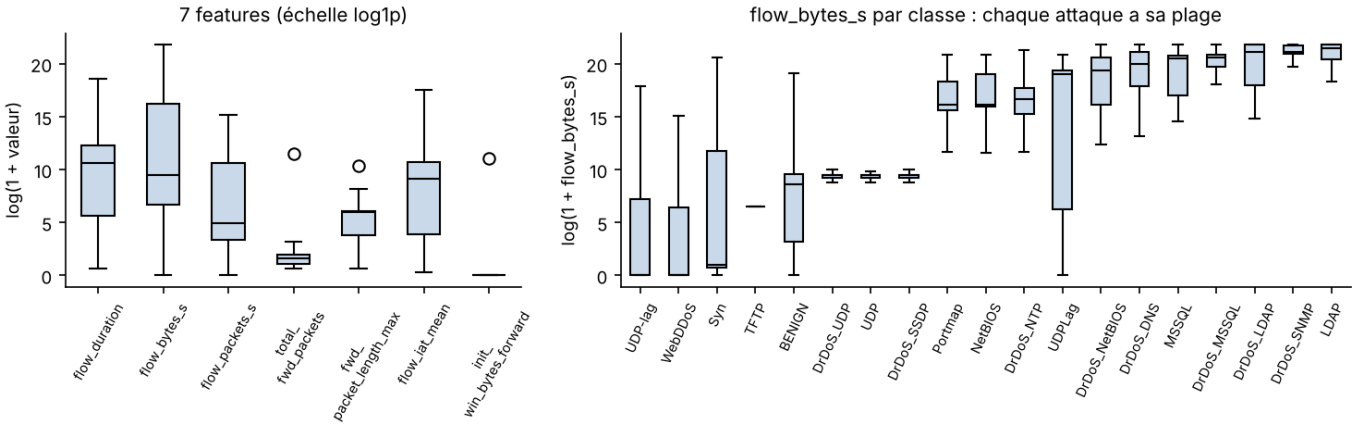
`nunique() == 1` et `VarianceThreshold(0)` donnent la même liste de **12 colonnes constantes**, toujours égales à 0 : 6 drapeaux jamais activés (`bwd_psh_flags`, `fwd_urg_flags`, `bwd_urg_flags`, `fin_flag_count`, `psh_flag_count`, `ece_flag_count`) et 6 statistiques « bulk » (`fwd/bwd_avg_bytes_bulk`, `_packets_bulk`, `_bulk_rate`). S'y ajoute une colonne **quasi constante** (seuil 99,9 %) : `syn_flag_count`, nulle dans 99,94 % des lignes, dont les 177 exceptions sont à 94 % du BENIGN. Elle ne permet même pas de distinguer l'attaque Syn. Une colonne constante a la même valeur pour toutes les classes : sa variance et son information mutuelle avec le label sont nulles. Elle alourdirait aussi les calculs, ferait diviser par zéro les scalers (écart-type nul) et produirait des NaN dans la matrice de Pearson. **13 colonnes supprimées : 74 → 61.**

Question 9 — Valeurs aberrantes (2 pts)

Énoncé : boîtes à moustaches pour au moins 5 features, comptage IQR, faut-il supprimer les valeurs aberrantes dans un IDS ?

Feature	Q1	Q3	Borne haute	Aberrantes	% lignes
flow_duration (µs)	279	216 597	541 074	63 261	20,3
flow_bytes_s	787	11 499 626	28 747 883	57 327	18,4
flow_packets_s	28	40 000	99 958	53 976	17,4
total_fwd_packets	2	6	12	58 662	18,9
fwd_packet_length_max	42	440	1 037	19 731	6,3
packet_length_mean	54	440	1 019	16 334	5,3
flow_iat_mean (µs)	46	43 227	107 999	59 346	19,1
init_win_bytes_forward	−1	−1	−1	76 970	24,7

Figure 4 – Boîtes à moustaches (Q9)



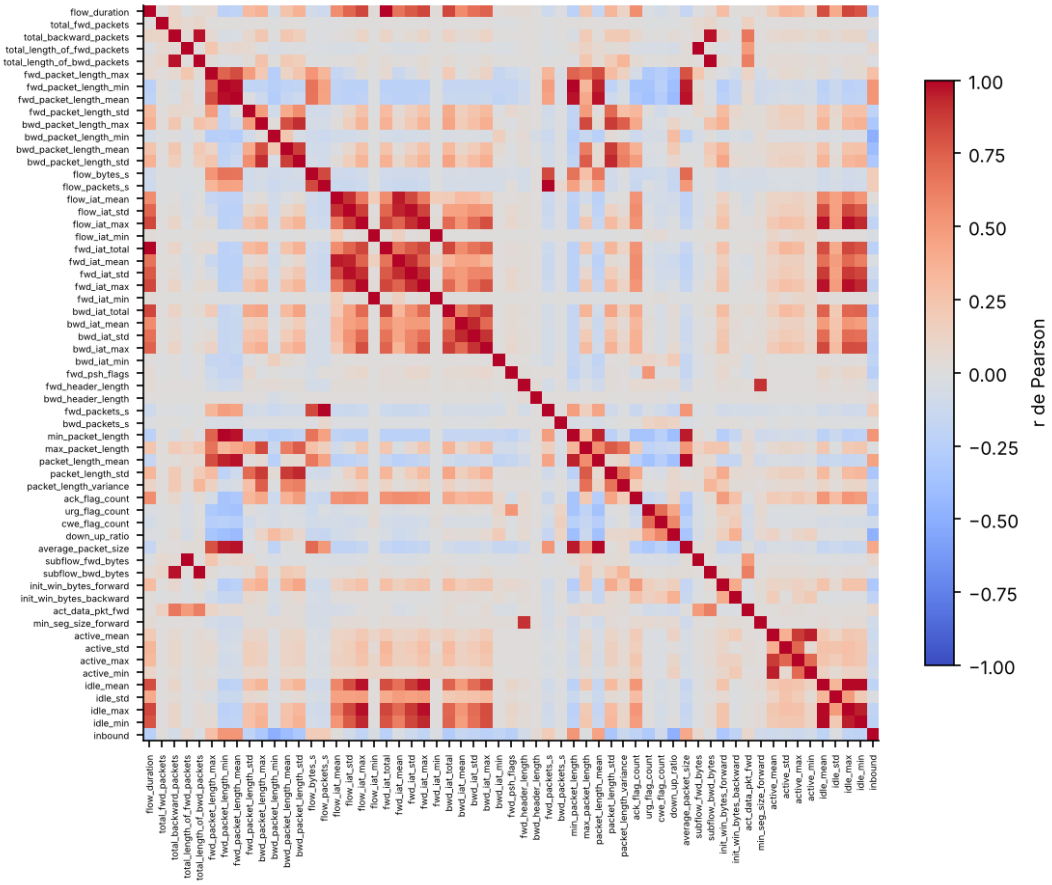
Au total, **188 382 lignes (60,6 %)** sont aberrantes sur au moins une feature, et la proportion atteint **100 % pour WebDDoS et Syn** (63 % pour BENIGN). Pour `init_win_bytes_forward`, Q1 = Q3 = −1 (valeur par défaut des flux UDP sans poignée de main) : l'IQR est nul, et toute vraie fenêtre TCP est déclarée aberrante. Cela montre les limites de l'IQR. **On ne supprime rien**, pour quatre raisons : (1) dans un IDS, l'anomalie est le signal à détecter, et la supprimer reviendrait à supprimer les attaques ; (2) l'IQR suppose une distribution unimodale, alors que chaque classe occupe sa propre plage (figure 4, à droite) ; (3) ce ne sont pas des erreurs de mesure, celles-ci ont déjà été traitées en Q6 ; (4) un modèle entraîné sans extrêmes raterait les attaques massives. Les extrêmes sont gérés par log1p et RobustScaler (Q12).

Partie D — Transformation (11 pts)

Question 10 — Corrélation et sélection de features (2 pts)

Énoncé : matrice de Pearson, supprimer une feature de chaque paire avec |r| > 0,95, liste retirée et nombre de features restantes.

Figure 5 – Matrice de corrélation de Pearson (59 features, avant sélection)



La matrice porte sur 59 features (`protocol`, catégorielle, est exclue). Elle montre des blocs quasi identiques : temps inter-arrivées, tailles de paquets, volumes. On trouve **25 paires avec |r| > 0,95**. En parcourant le triangle supérieur, on retire la seconde feature de chaque paire : **16 features retirées, 44 restantes** (311 062 × 45 avec le label).

Features retirées	Gardée (représentant)	|r|	Raison
subflow_fwd_bytes, subflow_bwd_bytes, total_length_of_bwd_packets	total_length_of_fwd_packets, total_backward_packets	0,97–1,00	Un sous-flux par flux ; paquets de taille fixe
fwd_iat_total, fwd_iat_mean / std / min / max, fwd_packets_s	flow_duration, flow_iat_*, flow_packets_s	0,96–1,00	Flux quasi unidirectionnels (aller ≈ total)
idle_mean, idle_max, idle_min	flow_iat_max	0,95–0,99	Inactivité ≈ plus grand écart entre paquets
min_packet_length, fwd_packet_length_mean, packet_length_mean, average_packet_size	fwd_packet_length_min	0,96–1,00	Paquets fixes : min ≈ moyenne

Ces redondances viennent de la construction des features par CICFlowMeter, renforcée par la nature unidirectionnelle des DDoS. Garder une seule feature par bloc évite la multicolinéarité (coefficients instables, importances diluées) sans perte d'information notable. Deux limites : un **effet de chaîne** (idle_min ou packet_length_mean sont retirées alors que leur partenaire l'est aussi) et le fait que Pearson ne capte que les liens **linéaires**. Une corrélation de Spearman ou une importance par forêt aléatoire serait un complément. La sélection, sans utilisation du label, présente un risque de fuite faible.

```
haut = corr.abs().where(np.triu(np.ones(corr.shape, dtype=bool), k=1))
a_retirer = [c for c in haut.columns if (haut[c] > 0.95).any()]   # 16 features
df = df.drop(columns=a_retirer)
```

Question 11 — Encodage (3 pts)

Énoncé : cibles binaire et multiclasse, encodage de protocol, justification LabelEncoder / One-Hot / ColumnTransformer.

Les noms d'une même attaque sont d'abord **harmonisés** (DrDoS_X + X → X, UDP-lag + UDPLag → UDPLag) : on passe de 19 noms bruts à **14 classes**, et le label d'origine est gardé dans `label_brut`. Cible binaire `y_bin` : 215 747 attaques (1) contre 95 315 BENIGN (0), soit 69 % / 31 %. Cible multiclasse `y_multi` : codes 0 à 13 (BENIGN = 0, ..., WebDDoS = 13). `protocol` (0, 6 = TCP, 17 = UDP) est encodée en One-Hot (`protocol_0` : 1 653, `protocol_6` : 76 970, `protocol_17` : 232 439). Les 5 autres colonnes binaires sont déjà en 0/1. Le jeu compte 46 features.

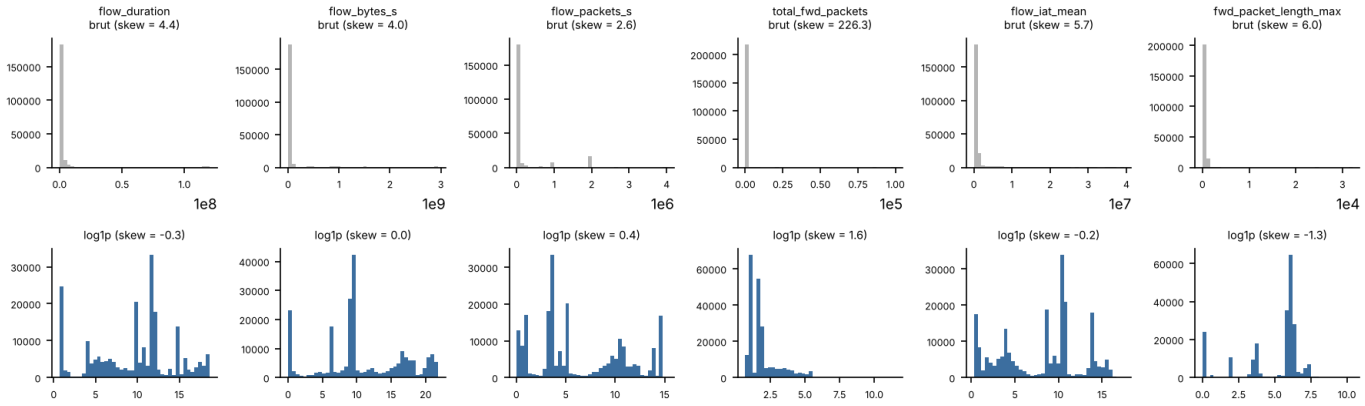
Méthode	Utilisée pour	Justification
LabelEncoder	La cible y_multi	Les classifieurs attendent une cible entière et ne traitent pas ces codes comme ordonnés. C'est l'usage prévu (cible uniquement).
One-Hot	La feature protocol	0/6/17 sont des identifiants IANA **sans ordre** : un encodage entier créerait un faux ordre et de fausses distances (KNN, SVM, modèles linéaires). Seulement 3 colonnes.
ColumnTransformer	Le pipeline final (Q16)	Applique un traitement par groupe de colonnes (One-Hot + scaler), ajusté sur le train seulement, et réutilisable sur de nouvelles données.

Question 12 — Normalisation (3 pts)

Énoncé : observer l'asymétrie (histogrammes, skew()), choisir un scaler et le justifier ; fit sur le train uniquement.

Le découpage 70/30 (Q13) est fait **avant** la normalisation, pour que le scaler ne voie jamais le test. 38 colonnes continues sont normalisées ; les 8 colonnes binaires restent en 0/1. Sur le train, les 38 features sont toutes très asymétriques (**médiane |skew| = 7,75**, jusqu'à 226 pour total_fwd_packets). Après log1p signé, sign(x)·log(1+|x|), la médiane tombe à **1,91**, et les histogrammes laissent voir un pic par type d'attaque.

Figure 6 – Distributions sur le train : brutes (haut) et après log1p (bas) (Q12)



flow_bytes_s (train)	Min	Médiane	99 %	Max	Constat
StandardScaler	−0,31	−0,31	5,52	5,72	Écrasé par les extrêmes
MinMaxScaler	0,00	0,00	0,97	1,00	Majorité collée à 0
RobustScaler seul	0,00	0,00	246,5	254,8	Traîne énorme
log1p + RobustScaler	−0,99	0,00	1,28	1,29	Centré, plage ≈ ±1

Choix : log1p signé puis RobustScaler. Le logarithme compresse des valeurs étalées sur plusieurs ordres de grandeur. RobustScaler (médiane et IQR) n'est pas sensible aux extrêmes, que l'on a choisi de garder en Q9 ; StandardScaler et MinMaxScaler seraient dominés par eux. Le scaler est ajusté sur `X_train` seulement : la médiane du train transformé vaut 0, celle du test 0,01, ce qui prouve que le test utilise les paramètres du train.

Question 13 — Découpage train/test (3 pts)

Énoncé : 70/30 stratifié sur Label, random_state = 42, tailles et répartition ; discuter l'alternative jour 1 / jour 2.

`train_test_split(test_size=0.30, stratify=label, random_state=42)` donne **217 743 lignes de train (70 %) et 93 319 de test (30 %)**, sans aucune ligne commune. L'écart maximal entre les proportions des classes du train et du test est de **0,001 point de %**, et le taux d'attaques vaut 69,36 % dans les deux ensembles. Même WebDDoS garde 124 exemples de test (BENIGN 30,64 %, UDP 16,46 %, NTP 15,14 % ... Portmap 0,25 %, WebDDoS 0,13 %).

Alternative jour 1 → jour 2	Classes	Problème
Seulement jour 1 (absentes du test)	DNS, NTP, SNMP, SSDP, TFTP, WebDDoS	6 classes sur 14 jamais évaluées
Seulement jour 2 (jamais vue en train)	Portmap	Classe inconnue : rappel multiclasse = 0 (confondue avec NetBIOS / LDAP)

Alternative jour 1 → jour 2	Classes	Problème
Les deux jours	BENIGN, Syn, LDAP, MSSQL, NetBIOS, UDP, UDPLag	Évaluables, mais avec une dérive de distribution (outils, IP, volumes)

Le scénario jour 1 / jour 2 est plus **réaliste** (train sur le passé, test sur le futur, avec une attaque inconnue). Il mesure la généralisation, alors que le découpage aléatoire mesure surtout l'interpolation entre flux d'un même jour. On garde le découpage stratifié demandé, qui rend les 14 classes évaluables, en sachant que ses scores seront **optimistes**.

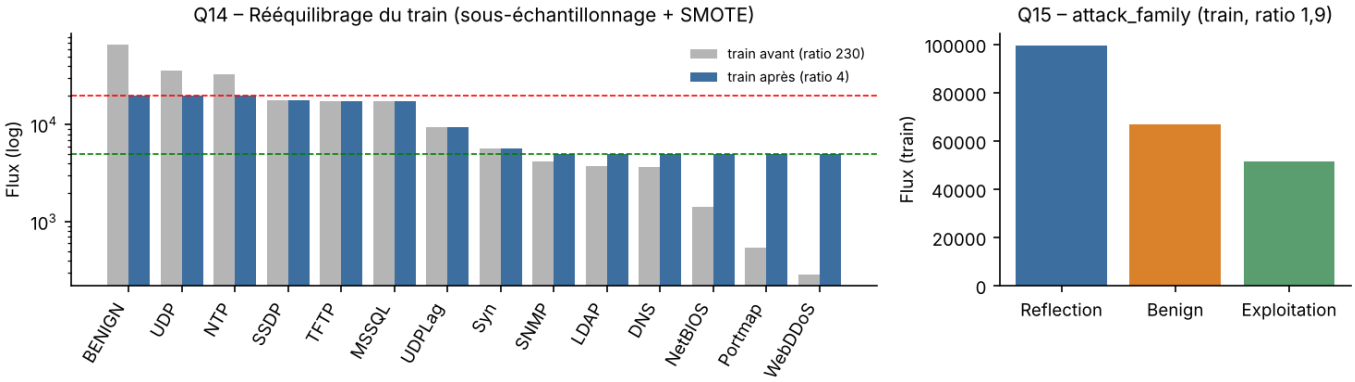
Partie E — Déséquilibre et familles d'attaques (6 pts)

Question 14 — Déséquilibre des classes (3 pts)

Énoncé : ratio majoritaire / minoritaire, stratégie sur le train seulement, graphique avant/après, pourquoi ne jamais rééquilibrer le test.

Sur le train, BENIGN compte 66 720 flux et WebDDoS 290 : **ratio = 230,1**. Stratégie hybride, **appliquée au train uniquement** : sous-échantillonnage aléatoire des classes de plus de 20 000 flux (BENIGN, UDP, NTP), puis SMOTE (k = 5) pour porter à 5 000 les classes de moins de 5 000 (SNMP, LDAP, DNS, NetBIOS, Portmap, WebDDoS). Les colonnes binaires sont arrondies à 0/1 après SMOTE. Le train passe de 217 743 à **158 367 lignes**, le ratio tombe à **4,0**, et le test reste inchangé (93 319 lignes).

Figure 7 – Déséquilibre des classes et familles d'attaques



Un sous-échantillonnage seul jusqu'à 290 flux par classe jetterait plus de 98 % du train ; un SMOTE seul jusqu'à 66 720 rendrait WebDDoS artificiel à 99,6 %. Le compromis limite les deux défauts. L'alternative `class_weight='balanced'` (poids de 0,23 pour BENIGN à 53,6 pour WebDDoS) ne modifie pas les données, mais n'est pas acceptée par tous les modèles. **Le test ne doit jamais être rééquilibré :** (1) il doit refléter la distribution réelle que verra l'IDS, et ses métriques (précision, taux de fausses alarmes) en dépendent ; (2) SMOTE sur le test ferait évaluer le modèle sur des flux inventés, ce qui gonfle les scores ; (3) sous-échantillonner jetterait des exemples réels ; (4) les plus proches voisins de SMOTE feraient passer de l'information du test vers le train.

Question 15 — Regroupement par familles (3 pts)

Énoncé : attack_family (Benign, Reflection, Exploitation) à partir des labels bruts, classement de WebDDoS justifié, rejouer 11 à 13 et comparer.

Reflection regroupe 12 labels bruts (DNS, LDAP, MSSQL, NTP, NetBIOS, SNMP, SSDP, TFTP, Portmap), et **Exploitation** regroupe Syn, UDP, UDP-lag, UDPLag et WebDDoS. **WebDDoS est classé en Exploitation** : c'est une inondation HTTP envoyée directement au serveur web, sans serveur tiers réflecteur, sans usurpation ni amplification, qui abuse d'un protocole légitime comme le fait Syn (son nom ne porte d'ailleurs pas le préfixe DrDoS). Étapes rejouées : encodage (Benign = 0, Exploitation = 1, Reflection = 2), découpage 70/30 stratifié sur la famille (217 743 / 93 319), puis log1p et RobustScaler réajustés sur ce nouveau train.

Cible	Classes	Plus grande (train)	Plus petite (train)	Ratio
Labels bruts	19	66 720 (BENIGN)	290 (WebDDoS)	230,1
Multiclasse harmonisée	14	66 720 (BENIGN)	290 (WebDDoS)	230,1
attack_family	3	99 615 (Reflection, 45,75 %)	51 408 (Exploitation, 23,61 %)	1,9

Le déséquilibre disparaît presque (230 → 1,9) : aucun rééquilibrage n'est nécessaire pour cette cible. Reflection devient majoritaire, et Benign garde 30,64 %. La stratification reste parfaite (mêmes proportions dans le train et le test). Les flux ambigus de la Q7 tombent presque tous dans la même famille, et les 3 familles existent les deux jours, ce qui rendrait le scénario jour 1 / jour 2 évaluable. Cette granularité suffit souvent à choisir la contre-mesure

(filtrer les réponses réfléchies ou protéger la pile TCP / le serveur web).

Partie F — Pipeline et export (3 pts)

Question 16 — Pipeline reproductible (3 pts)

Énoncé : Pipeline / ColumnTransformer, sauvegarde joblib, export Parquet, vérification des dimensions après rechargement.

```
pretraitement = ColumnTransformer([
    ("continues", Pipeline([("log1p", FunctionTransformer(log1p_signe, feature_names_out="one-to-one")),
                            ("scaler", RobustScaler())]), CONTINUES),              # 38 colonnes
    ("protocole", OneHotEncoder(handle_unknown="ignore", sparse_output=False), ["protocol"]),
    ("binaires", "passthrough", BIN_COLS)], remainder="drop")                      # 5 colonnes 0/1
pipeline = Pipeline([("pretraitement", pretraitement)]).fit(X_brut_train)       # TRAIN seulement
joblib.dump(pipeline, f"{EXPORT}/pipeline_pretraitement.joblib")
train_export.to_parquet(f"{EXPORT}/train.parquet", index=False)
```

Le pipeline prend les colonnes brutes retenues (avec `protocol` d'origine) et produit **46 colonnes**. L'écart maximal avec le prétraitement manuel des Q11-Q12 est de **3,8 × 10^{-6}**, l'arrondi du float32. Le rééquilibrage n'est volontairement pas inclus, car il ne doit jamais toucher au test ; on l'ajouterait avec un `imblearn.pipeline.Pipeline`. Fichiers exportés : `pipeline_pretraitement.joblib` (0,01 Mo), `train.parquet` (11,88 Mo) et `test.parquet` (5,25 Mo), qui contiennent les 46 features et 5 colonnes cibles (label, label_brut, attack_family, y_bin, y_multi). **Vérification après rechargement :** train 217 743 × 51 et test 93 319 × 51, identiques ; le contenu est égal (`DataFrame.equals`), et le pipeline rechargé transforme le test exactement comme l'original (`np.allclose`). Parquet est colonnaire, compressé et conserve les types ; joblib sérialise efficacement les objets scikit-learn. Pour recharger le pipeline, il faut définir `log1p_signe` et utiliser scikit-learn 1.6.1.

Tableau récapitulatif — taille du jeu après chaque étape

Étape	Opération	Lignes	Colonnes
Données brutes	18 CSV CIC-DDoS2019	70 427 637	88
Q3	Échantillonnage stratifié plafonné, float32 / int32	916 769	88
Q4	Doublons de colonnes, snake_case	916 769	82
Q5	Colonnes identifiantes supprimées	916 769	74
Q6	Lignes NaN / ±inf supprimées	889 188	74
Q7	Lignes dupliquées supprimées	311 062	74
Q8	Colonnes constantes / quasi constantes supprimées	311 062	61
Q9	Valeurs aberrantes conservées	311 062	61
Q10	Features corrélées (|r| > 0,95) supprimées	311 062	45
Q11	One-Hot protocol, cibles y_bin / y_multi, label_brut	311 062	46 features + 4
Q12-Q13	Découpage 70/30 stratifié, log1p + RobustScaler	train 217 743 / test 93 319	46
Q14	Sous-échantillonnage + SMOTE (train seulement)	train 158 367 / test 93 319	46
Q15	Cible attack_family (3 familles)	train 217 743 / test 93 319	46
Q16	Export Parquet (features + 5 cibles)	train 217 743 / test 93 319	51

Référence : I. Sharafaldin, A. H. Lashkari, S. Hakak, A. A. Ghorbani, « Developing Realistic Distributed Denial of Service (DDoS) Attack Dataset and Taxonomy », IEEE 53rd International Carnahan Conference on Security Technology, Chennai, 2019.

Annexe — Code (version condensée du notebook, exécutable dans l'ordre)

Le notebook complet, avec toutes les sorties, les graphiques et les commentaires, est fourni dans l'archive : `Projet2_Pretraitement_CICDDoS2019_TjeckTjeckThomasNoel.ipynb`. Le code ci-dessous en reprend toutes les étapes de traitement, sans les affichages secondaires.

```
# ===== Imports =====
import pandas as pd
pd.options.mode.chained_assignment = None
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt
import time, csv, os, glob
```

```
# ===== Q1 — Téléchargement (version Kaggle des CSV officiels CIC) =====
import kagglehub
path = kagglehub.dataset_download("rodrigorosasilva/cic-ddos2019-30gb-full-dataset-csv-files")
csvs = sorted(glob.glob(os.path.join(path, "**", "*.csv"), recursive=True))
for f in csvs:
    print(f"{os.path.getsize(f)/1e9:6.2f} Go  {os.path.relpath(f, path)}")
```

```
# ===== Q2 — Taxonomie : effectifs réels (lecture de la seule colonne Label) =====
from collections import Counter
compte = Counter()
for f in csvs:
    for chunk in pd.read_csv(f, usecols=lambda c: c.strip() == "Label", chunksize=1_000_000):
        compte.update(chunk.iloc[:, 0].str.strip())
labels = pd.Series(compte).sort_values(ascending=False)
print(labels)
labels.plot(kind="bar", logy=True, figsize=(12, 5), title="Distribution des classes (échelle log)")
plt.ylabel("Nombre de flux (log)"); plt.tight_layout(); plt.show()
```

```
# ===== Q3 — Chargement par morceaux + échantillonnage stratifié plafonné =====
np.random.seed(42)
rng = np.random.default_rng(42)
MAX_PAR_CLASSE = 50_000
CHUNKSIZE = 500_000
taux = {lab: min(1.0, MAX_PAR_CLASSE / n) for lab, n in labels.items()}
taux["BENIGN"] = 1.0                                  # on garde tout le trafic normal
morceaux = []
for f in csvs:
    for chunk in pd.read_csv(f, chunksize=CHUNKSIZE, low_memory=False):
        chunk.columns = chunk.columns.str.strip()      # " Label" -> "Label"
        chunk["Label"] = chunk["Label"].str.strip()
        p = chunk["Label"].map(taux).fillna(1.0).to_numpy()
        morceaux.append(chunk[rng.random(len(chunk)) < p])   # tirage stratifié
df = pd.concat(morceaux, ignore_index=True); del morceaux
mem_avant = df.memory_usage(deep=True).sum() / 1e6
for col in df.select_dtypes("float64").columns:
    df[col] = df[col].astype("float32")
for col in df.select_dtypes("int64").columns:
    df[col] = df[col].astype("int32")
mem_apres = df.memory_usage(deep=True).sum() / 1e6
print(f"Mémoire : {mem_avant:,.1f} Mo -> {mem_apres:,.1f} Mo ({100*(1-mem_apres/mem_avant):.1f} %)")
```

```
# ===== Q3 — Cache Parquet dans Google Drive (évite de relire 31 Go) =====
from google.colab import drive
drive.mount("/content/drive")
CACHE = "/content/drive/MyDrive/CICDDoS2019_cache"
os.makedirs(CACHE, exist_ok=True)
obj_cols = df.select_dtypes("object").columns              # Parquet : un seul type par colonne
df[obj_cols] = df[obj_cols].astype(str)
df.to_parquet(f"{CACHE}/echantillon_q3.parquet", index=False)
labels.to_frame("n").to_parquet(f"{CACHE}/labels_q2.parquet")
# Rechargement rapide après redémarrage :
# df = pd.read_parquet(f"{CACHE}/echantillon_q3.parquet"); labels = pd.read_parquet(f"{CACHE}/labels_q2.parquet")["n"]
```

```
# ===== Q4 — Nettoyage des noms de colonnes =====
import re
df.columns = df.columns.str.strip()
doublons_nom = [c for c in df.columns if re.search(r"\.\d+$", c)]
non_constantes = [c for c in df.columns if df[c].nunique(dropna=False) > 1]   # constantes : Q8
hashes = {c: pd.util.hash_pandas_object(df[c], index=False).sum() for c in non_constantes}
vus, doublons_contenu = {}, []
for c, h in hashes.items():
    if h in vus and df[c].equals(df[vus[h]]):
        doublons_contenu.append((c, vus[h]))
    else:
        vus.setdefault(h, c)
a_supprimer = sorted({c for c, _ in doublons_contenu} | set(doublons_nom))
df = df.drop(columns=a_supprimer)
def snake(nom):
    nom = re.sub(r"[^0-9a-zA-Z]+", "_", nom.strip())
    nom = re.sub(r"([a-z])([A-Z])", r"\1_\2", nom)
    return re.sub(r"_+", "_", nom).strip("_").lower()
df.columns = [snake(c) for c in df.columns]
assert df.columns.is_unique
```

```
# ===== Q5 — Colonnes identifiantes (fuite de données) =====
ID_COLS = ["unnamed_0", "flow_id", "source_ip", "destination_ip",
           "source_port", "destination_port", "timestamp", "simillar_http"]
for col in ["source_ip", "timestamp", "source_port"]:
    pur = (df.groupby(col)["label"].nunique() == 1).mean()
    print(f"{col}: {df[col].nunique():,} valeurs, {100*pur:.1f} % liées à un seul label")
print(df.groupby("label")["source_port"].agg(lambda s: s.mode().iat[0]))
df = df.drop(columns=[c for c in ID_COLS if c in df.columns])
```

```
# ===== Q6 — NaN et valeurs infinies =====
num_cols = df.select_dtypes("number").columns
diag = pd.DataFrame({"NaN": df[num_cols].isna().sum(),
                     "+inf": np.isposinf(df[num_cols]).sum(),
                     "-inf": np.isneginf(df[num_cols]).sum()})
diag["total"] = diag.sum(axis=1)
diag_pb = diag[diag["total"] > 0]
lignes_pb = df[num_cols].isna().any(axis=1) | np.isinf(df[num_cols]).any(axis=1)
print("flow_duration == 0 :", (df.loc[lignes_pb, "flow_duration"] == 0).sum(), "/", lignes_pb.sum())
fig, axes = plt.subplots(1, 2, figsize=(15, 5))
diag_pb[["NaN", "+inf", "-inf"]].plot(kind="barh", stacked=True, ax=axes[0])
pb = df[diag_pb.index].isna() | np.isinf(df[diag_pb.index])
sns.heatmap(100 * pb.groupby(df["label"]).mean(), annot=True, fmt=".1f", cmap="Reds", ax=axes[1])
plt.tight_layout(); plt.show()
df = df.replace([np.inf, -np.inf], np.nan).dropna().reset_index(drop=True)
```

```
# ===== Q7 — Doublons =====
n_avant = df["label"].value_counts()
print("Lignes dupliquées :", df.duplicated().sum())
features = [c for c in df.columns if c != "label"]
memes = df.duplicated(subset=features, keep=False)
print("Groupes ambigus :", int((df[memes].groupby(features, sort=False)["label"].nunique() > 1).sum()))
df = df.drop_duplicates(keep="first").reset_index(drop=True)
tab = pd.DataFrame({"avant": n_avant, "après": df["label"].value_counts()}).fillna(0).astype(int)
tab["% perdu"] = (100 * (tab["avant"] - tab["après"]) / tab["avant"]).round(1)
print(tab.sort_values("% perdu", ascending=False))
```

```
# ===== Q8 — Colonnes constantes / quasi constantes =====
from sklearn.feature_selection import VarianceThreshold
X_num = df.drop(columns="label").select_dtypes("number")
nuniq = X_num.nunique()
constantes = nuniq[nuniq == 1].index.tolist()
vt = VarianceThreshold(threshold=0.0).fit(X_num)
assert set(constantes) == set(X_num.columns[~vt.get_support()])
part_dom = X_num.apply(lambda s: s.value_counts(normalize=True).iat[0])
quasi = part_dom[(part_dom >= 0.999) & (nuniq > 1)]
df = df.drop(columns=constantes + quasi.index.tolist())
```

```
# ===== Q9 — Valeurs aberrantes (IQR), conservées =====
FEATS = ["flow_duration", "flow_bytes_s", "flow_packets_s", "total_fwd_packets",
         "fwd_packet_length_max", "packet_length_mean", "flow_iat_mean", "init_win_bytes_forward"]
def bornes_iqr(s, k=1.5):
    q1, q3 = s.quantile([0.25, 0.75]); iqr = q3 - q1
    return q1 - k * iqr, q3 + k * iqr
masque = pd.Series(False, index=df.index)
for c in FEATS:
    bas, haut = bornes_iqr(df[c]); m = (df[c] < bas) | (df[c] > haut); masque |= m
    print(f"{c:25s} {m.sum():>7,} aberrantes ({100*m.mean():.1f} %)")
print((100 * masque.groupby(df["label"]).mean()).sort_values(ascending=False))
fig, axes = plt.subplots(2, 4, figsize=(18, 8))
for ax, c in zip(axes.ravel(), FEATS):
    sns.boxplot(y=np.log1p(df[c].clip(lower=0)), ax=ax); ax.set_title(c)
plt.tight_layout(); plt.show()
# Aucune ligne supprimée : les extrêmes sont la signature des attaques.
```

```
# ===== Q10 — Corrélation de Pearson et sélection =====
feat_corr = [c for c in df.select_dtypes("number").columns if c != "protocol"]
corr = df[feat_corr].corr(method="pearson")
sns.heatmap(corr, cmap="coolwarm", vmin=-1, vmax=1, center=0); plt.show()
haut = corr.abs().where(np.triu(np.ones(corr.shape, dtype=bool), k=1))
a_retirer = [c for c in haut.columns if (haut[c] > 0.95).any()]
df = df.drop(columns=a_retirer)
print(len(a_retirer), "retirées ;", df.shape[1] - 1, "features restantes")
```

```
# ===== Q11 — Encodage : harmonisation, cibles binaire / multiclasse, One-Hot protocol =====
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
df["label_brut"] = df["label"]
HARMONISATION = {"DrDoS_DNS": "DNS", "DrDoS_LDAP": "LDAP", "DrDoS_MSSQL": "MSSQL", "DrDoS_NTP": "NTP",
                 "DrDoS_NetBIOS": "NetBIOS", "DrDoS_SNMP": "SNMP", "DrDoS_SSDP": "SSDP",
                 "DrDoS_UDP": "UDP", "UDP-lag": "UDPLag"}
df["label"] = df["label"].replace(HARMONISATION)
df["y_bin"] = (df["label"] != "BENIGN").astype("int8")
le = LabelEncoder(); df["y_multi"] = le.fit_transform(df["label"]).astype("int8")
ohe = OneHotEncoder(sparse_output=False, dtype="int8", handle_unknown="ignore")
proto = ohe.fit_transform(df[["protocol"]])
proto_cols = [f"protocol_{int(v)}" for v in ohe.categories_[0]]
df = pd.concat([df.drop(columns="protocol"), pd.DataFrame(proto, columns=proto_cols, index=df.index)], axis=1)
FEATURES = [c for c in df.columns if c not in ("label", "label_brut", "y_bin", "y_multi")]
```

```
# ===== Q12 — Découpage puis normalisation log1p + RobustScaler (fit sur le train) =====
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler
train_idx, test_idx = train_test_split(df.index, test_size=0.30, stratify=df["label"], random_state=42)
X_train, X_test = df.loc[train_idx, FEATURES].copy(), df.loc[test_idx, FEATURES].copy()
BINAIRES = [c for c in FEATURES if X_train[c].nunique() <= 2]
CONTINUES = [c for c in FEATURES if c not in BINAIRES]
def log1p_signe(a):
    return np.sign(a) * np.log1p(np.abs(a))
print(X_train[CONTINUES].skew().abs().median(), log1p_signe(X_train[CONTINUES].astype("float64")).skew().abs().median())
scaler = RobustScaler().fit(log1p_signe(X_train[CONTINUES].astype("float64")))     # TRAIN seulement
for part in (X_train, X_test):
    part[CONTINUES] = scaler.transform(log1p_signe(part[CONTINUES].astype("float64"))).astype("float32")
```

```
# ===== Q13 — Vérification du découpage et alternative jour 1 / jour 2 =====
y_train_bin, y_test_bin = df.loc[train_idx, "y_bin"], df.loc[test_idx, "y_bin"]
y_train_multi, y_test_multi = df.loc[train_idx, "y_multi"], df.loc[test_idx, "y_multi"]
lab_train, lab_test = df.loc[train_idx, "label"], df.loc[test_idx, "label"]
assert set(train_idx).isdisjoint(test_idx)
rep = pd.DataFrame({"train (%)": 100 * lab_train.value_counts(normalize=True),
                    "test (%)": 100 * lab_test.value_counts(normalize=True)})
print(rep, (rep["train (%)"] - rep["test (%)"]).abs().max())
```

```
JOUR1 = {"DrDoS_DNS", "DrDoS_LDAP", "DrDoS_MSSQL", "DrDoS_NTP", "DrDoS_NetBIOS", "DrDoS_SNMP",
         "DrDoS_SSDP", "DrDoS_UDP", "TFTP", "UDP-lag", "WebDDoS"}
JOUR2 = {"LDAP", "MSSQL", "NetBIOS", "Portmap", "UDP", "UDPLag"}       # Syn et BENIGN : les deux jours

# ===== Q14 — Rééquilibrage du TRAIN uniquement =====
from imblearn.under_sampling import RandomUnderSampler
from imblearn.over_sampling import SMOTE
from sklearn.utils.class_weight import compute_class_weight
cnt = y_train_multi.value_counts()
print("Ratio max/min :", cnt.max() / cnt.min())
rus = RandomUnderSampler(sampling_strategy={c: 20_000 for c, n in cnt.items() if n > 20_000}, random_state=42)
X_tmp, y_tmp = rus.fit_resample(X_train, y_train_multi)
cnt2 = pd.Series(y_tmp).value_counts()
smote = SMOTE(sampling_strategy={c: 5_000 for c, n in cnt2.items() if n < 5_000}, k_neighbors=5, random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_tmp, y_tmp)
X_train_res[BINAIRES] = X_train_res[BINAIRES].round().clip(0, 1)
cw = compute_class_weight("balanced", classes=np.unique(y_train_multi), y=y_train_multi)   # alternative

# ===== Q15 — Familles d'attaques =====
REFLECTION = {"DrDoS_DNS", "DrDoS_LDAP", "LDAP", "DrDoS_MSSQL", "MSSQL", "DrDoS_NTP",
              "DrDoS_NetBIOS", "NetBIOS", "DrDoS_SNMP", "DrDoS_SSDP", "TFTP", "Portmap"}
EXPLOITATION = {"Syn", "DrDoS_UDP", "UDP", "UDP-lag", "UDPLag", "WebDDoS"}
df["attack_family"] = df["label_brut"].map(
    lambda l: "Benign" if l == "BENIGN" else ("Reflection" if l in REFLECTION else "Exploitation"))
le_fam = LabelEncoder(); df["y_family"] = le_fam.fit_transform(df["attack_family"]).astype("int8")
tr_f, te_f = train_test_split(df.index, test_size=0.30, stratify=df["attack_family"], random_state=42)
Xf_train, Xf_test = df.loc[tr_f, FEATURES].copy(), df.loc[te_f, FEATURES].copy()
scaler_f = RobustScaler().fit(log1p_signe(Xf_train[CONTINUES].astype("float64")))
for part in (Xf_train, Xf_test):
    part[CONTINUES] = scaler_f.transform(log1p_signe(part[CONTINUES].astype("float64"))).astype("float32")

# ===== Q16 — Pipeline, joblib et Parquet =====
import joblib
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import FunctionTransformer
PROTO_COLS = [c for c in FEATURES if c.startswith("protocol_")]
BIN_COLS = [c for c in BINAIRES if c not in PROTO_COLS]
X_brut = df[CONTINUES + BIN_COLS].copy()
X_brut["protocol"] = df[PROTO_COLS].idxmax(axis=1).str.replace("protocol_", "").astype(int)
X_brut_train, X_brut_test = X_brut.loc[train_idx], X_brut.loc[test_idx]
pretraitement = ColumnTransformer([
    ("continues", Pipeline([("log1p", FunctionTransformer(log1p_signe, feature_names_out="one-to-one")),
                            ("scaler", RobustScaler())]), CONTINUES),
    ("protocole", OneHotEncoder(handle_unknown="ignore", sparse_output=False, dtype="int8"), ["protocol"]),
    ("binaires", "passthrough", BIN_COLS),
], remainder="drop", verbose_feature_names_out=False)
pipeline = Pipeline([("pretraitement", pretraitement)]).fit(X_brut_train)      # TRAIN seulement
noms = pipeline.get_feature_names_out()
Xp_train = pd.DataFrame(pipeline.transform(X_brut_train), columns=noms, index=train_idx).astype("float32")
Xp_test = pd.DataFrame(pipeline.transform(X_brut_test), columns=noms, index=test_idx).astype("float32")
EXPORT = f"{CACHE}/export"; os.makedirs(EXPORT, exist_ok=True)
joblib.dump(pipeline, f"{EXPORT}/pipeline_pretraitement.joblib")
CIBLES = ["label", "label_brut", "attack_family", "y_bin", "y_multi"]
train_export = pd.concat([Xp_train, df.loc[train_idx, CIBLES]], axis=1).reset_index(drop=True)
test_export = pd.concat([Xp_test, df.loc[test_idx, CIBLES]], axis=1).reset_index(drop=True)
train_export.to_parquet(f"{EXPORT}/train.parquet", index=False)
test_export.to_parquet(f"{EXPORT}/test.parquet", index=False)
train_rel, test_rel = pd.read_parquet(f"{EXPORT}/train.parquet"), pd.read_parquet(f"{EXPORT}/test.parquet")
assert train_rel.shape == train_export.shape and test_rel.shape == test_export.shape
assert train_rel.equals(train_export) and test_rel.equals(test_export)
pipe_rel = joblib.load(f"{EXPORT}/pipeline_pretraitement.joblib")
assert np.allclose(pipe_rel.transform(X_brut_test), Xp_test.to_numpy(), atol=1e-5)
```